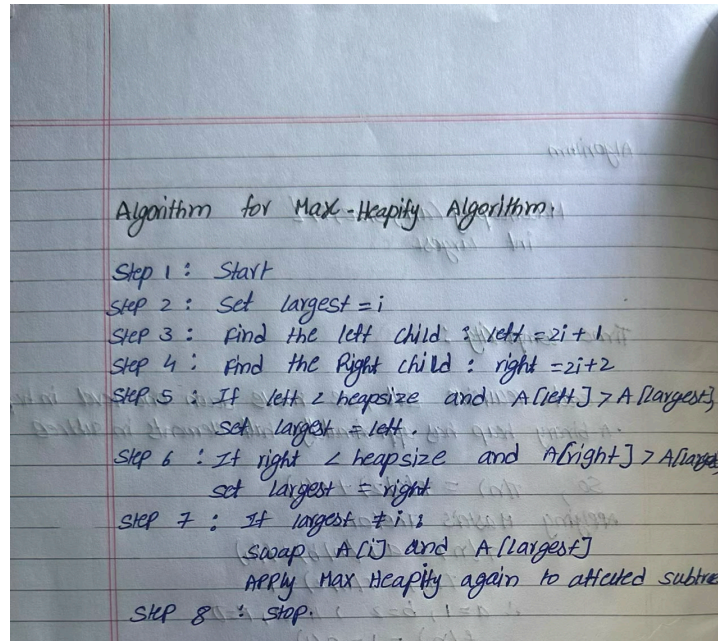


PRACTICAL - 3

AIM : Implementation of max-heap sort algorithm

ALGORITHM : // Heap sort



PROGRAM : // Heap sort

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;
```



```
if (largest != i)
{
    swap(arr[i], arr[largest]);

    heapify(arr, n, largest);
}
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);

        heapify(arr, i, 0);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    heapSort(arr, n);
}
```

```
cout << "Sorted array: ";
for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}
```

OUTPUT:

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB on  master
● > ./HeapSort
Enter number of elements: 4
Enter 4 elements: 21 09 06 07
Sorted array: 6 7 9 21 %
```

ALGORITHM :

Heapify method

```

Algorithm
Max_Heapify(arr, size, i) {
    int largest = i;
    int L = (2*i);
    int R = (2*i+1);
    while (L < size && arr[L] > arr[largest])
        largest = L;
    while (R < size && arr[R] > arr[largest])
        largest = R;
    if (largest != i) {
        swap(arr[largest], arr[i]);
        Max_Heapify(arr, size, largest);
    }
}

Heap_Sort(i = size/2, i >= 1; i--) {
    Max_Heapify(arr, size, i);
}

for (i = size; i >= 1; i--) {
    swap(arr[1], arr[i]);
    Max_Heapify(arr, size, 1);
}

T.C => O(n) * O(log n)
O(n log n)
    
```



TIME COMPLEXITY : // Heap sort

Time complexity :

- each recursive call, we move down one level in heap
- A binary heap has approximately half elements in subtree

So, $T(n) = T(n/2) + O(1)$

Applying Master's Theorem :

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = T(n/2) + 1$$

$$\therefore a=1, b=2, K=0, P=0$$

$$f(n) = 1 = O(1)$$

By calculating $\Rightarrow n^{\log_b a}$

$$= n^{\log_2 1}$$

$$(\because \log_2 1 = 0) \quad n^0 = 1$$

So, $f(n) = O(n^{\log_b a})$

$$T(n) = O(\log n) \Rightarrow \text{Master Theorem}$$

Heapify = $O(\log n)$

$\therefore$ Heap sort has n heapify operations

$$T(n) = n \times O(\log n)$$

$$T(n) = O(n \log n)$$

For, Max-heap sort :

Best = Average = worst = $O(n \log n)$

worst case +

$$T_{\text{worst}}(n) = T(n/2) + O(1)$$

$$a=1, b=2, f(n)=1$$

$$n^{\log_b a} = 1$$

$$\therefore T_{\text{worst}}(n) = O(n \log n)$$

Average case +

$$T_{\text{avg}}(n) = T(n/2) + O(1)$$

$$a=1, b=2, f(n)=n$$

$$n^{\log_2 1} = 1$$

$$\therefore T_{\text{avg}}(n) = O(n \log n)$$



CONCLUSION : Heap Sort is an efficient comparison-based sorting algorithm that uses a binary heap data structure to arrange elements. It first builds a max heap and then repeatedly moves the largest element to its correct position. Heap Sort has a time complexity of $O(n \log n)$, making it suitable for sorting large datasets. It also does not require extra arrays for sorting, making it a useful and efficient sorting technique.